Práticas de Inovação VI



Inferência com imagens

Edge Impulse + Arduino IDE

Pré-etapa) Configurar ambiente Python no VS Code

- Abra o VSCode e o Terminal integrado (PowerShell).
- Crie e ative um ambiente virtual (Windows):
  - py -3.13 -m venv .venv ou python -m venv .venv
  - Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
  - .\.venv\Scripts\Activate.ps1
  - python -m pip install --upgrade pip
  - python -m pip install pyserial numpy opencv-python pillow

1) Dataset de imagens

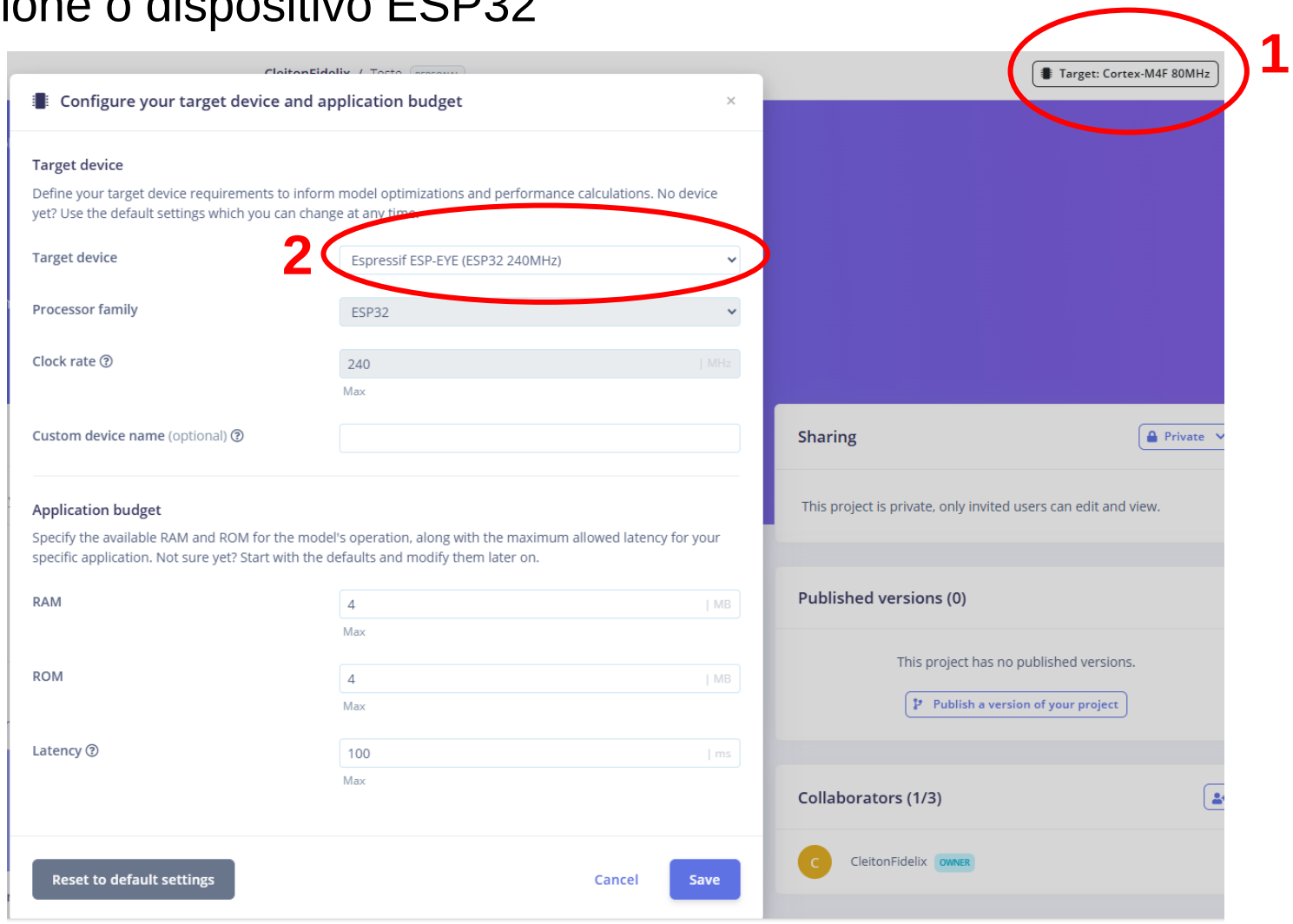
- Criar imagens com resolução 96×96 e grayscale.
- Faça ≥ 50 imagens por classe, equilibradas.
- Nome dos arquivos: classe. (1).jpg, classe. (2).jpg, ... (sem bounding boxes).
- Pasta por classe é ótima para upload em lote.





2) Edge Impulse (treino e exportação)

- Selecione o dispositivo ESP32

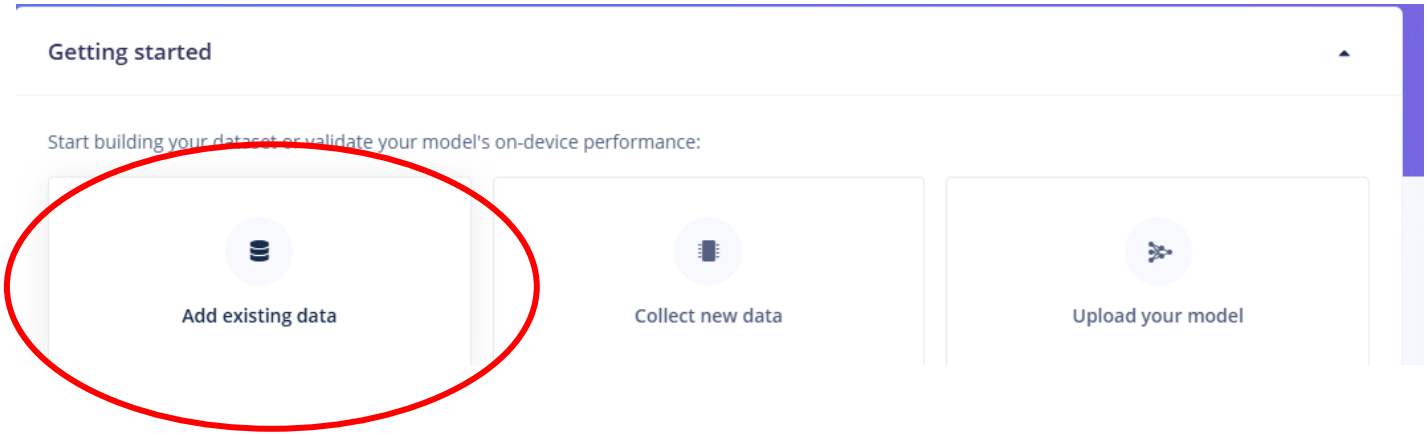


2) Edge Impulse (treino e exportação)

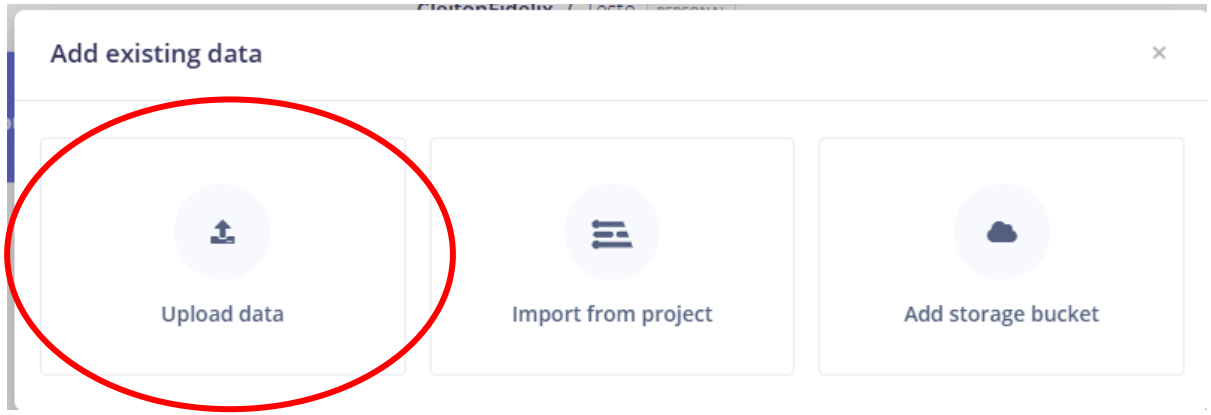
Adicione as imagens

extrair dataset.zip antes do upload

- 1)

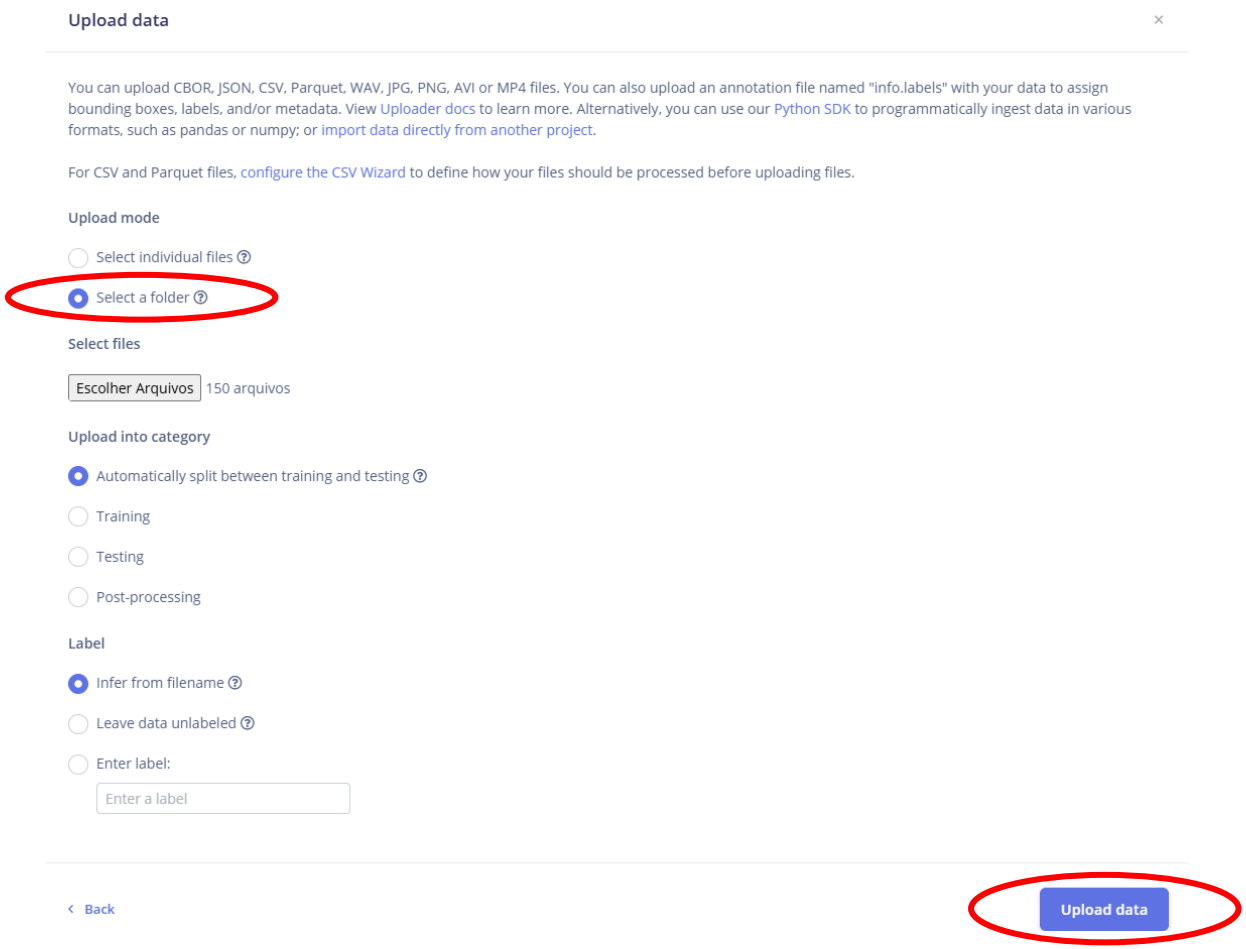


- 2)



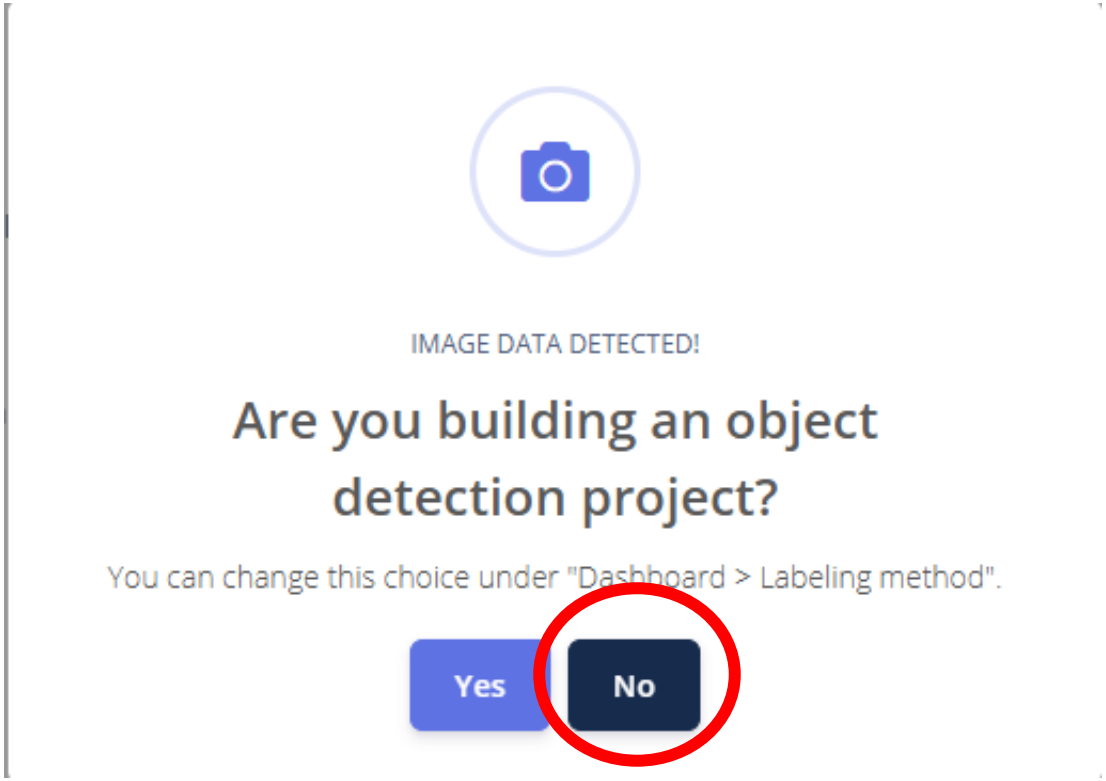
2) Edge Impulse (treino e exportação)

- Nomear os arquivos por classe: classe. (1).jpg, classe. (2).jpg, ... (sem bounding boxes).



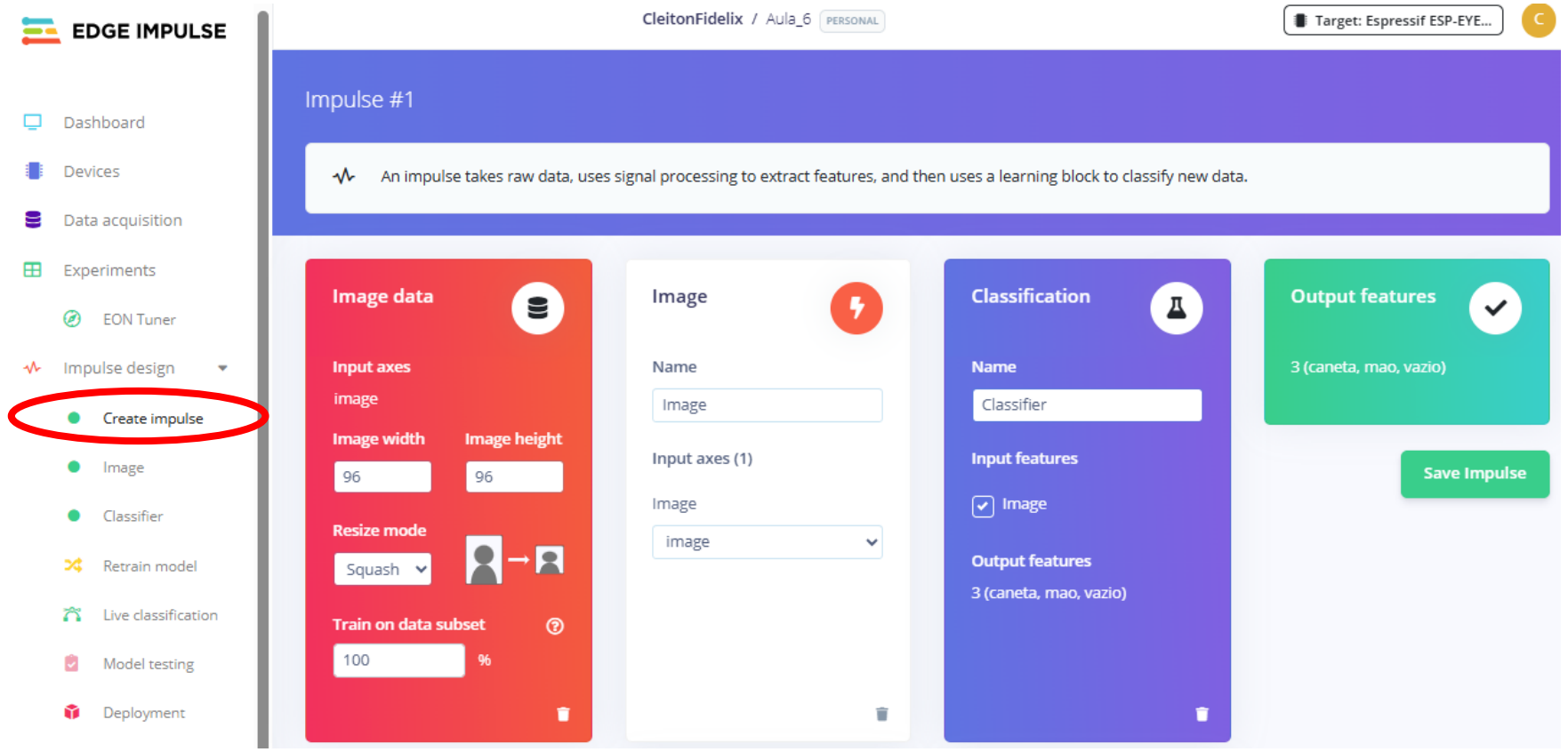
2) Edge Impulse (treino e exportação)

- Se estiver trabalhando com classificação de imagens com um único rótulo, sem caixas delimitadoras (sem *bounding boxes*), selecione não.



2) Edge Impulse (treino e exportação)

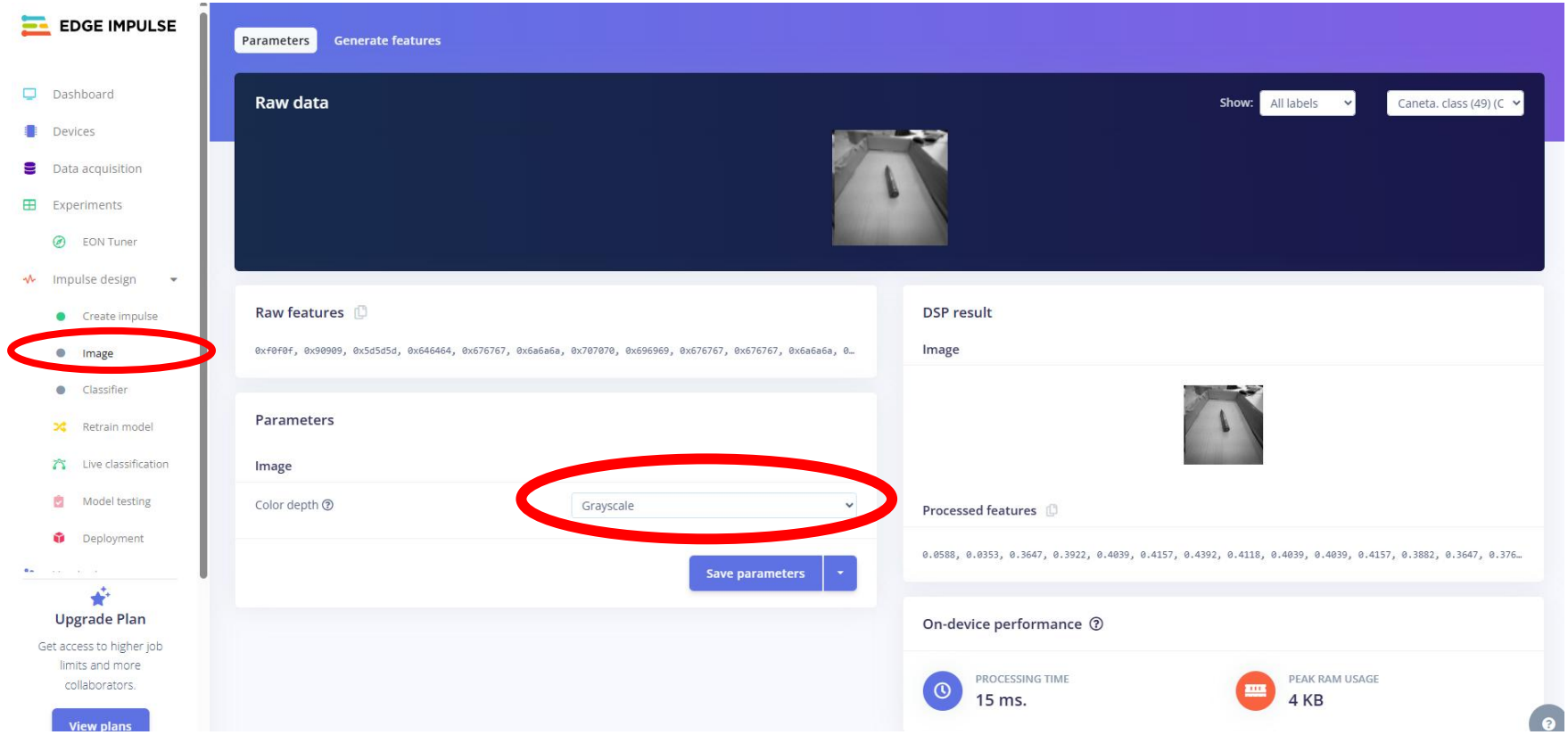
- Criar um impulso para imagem com ***Classification***



2) Edge Impulse (treino e exportação)

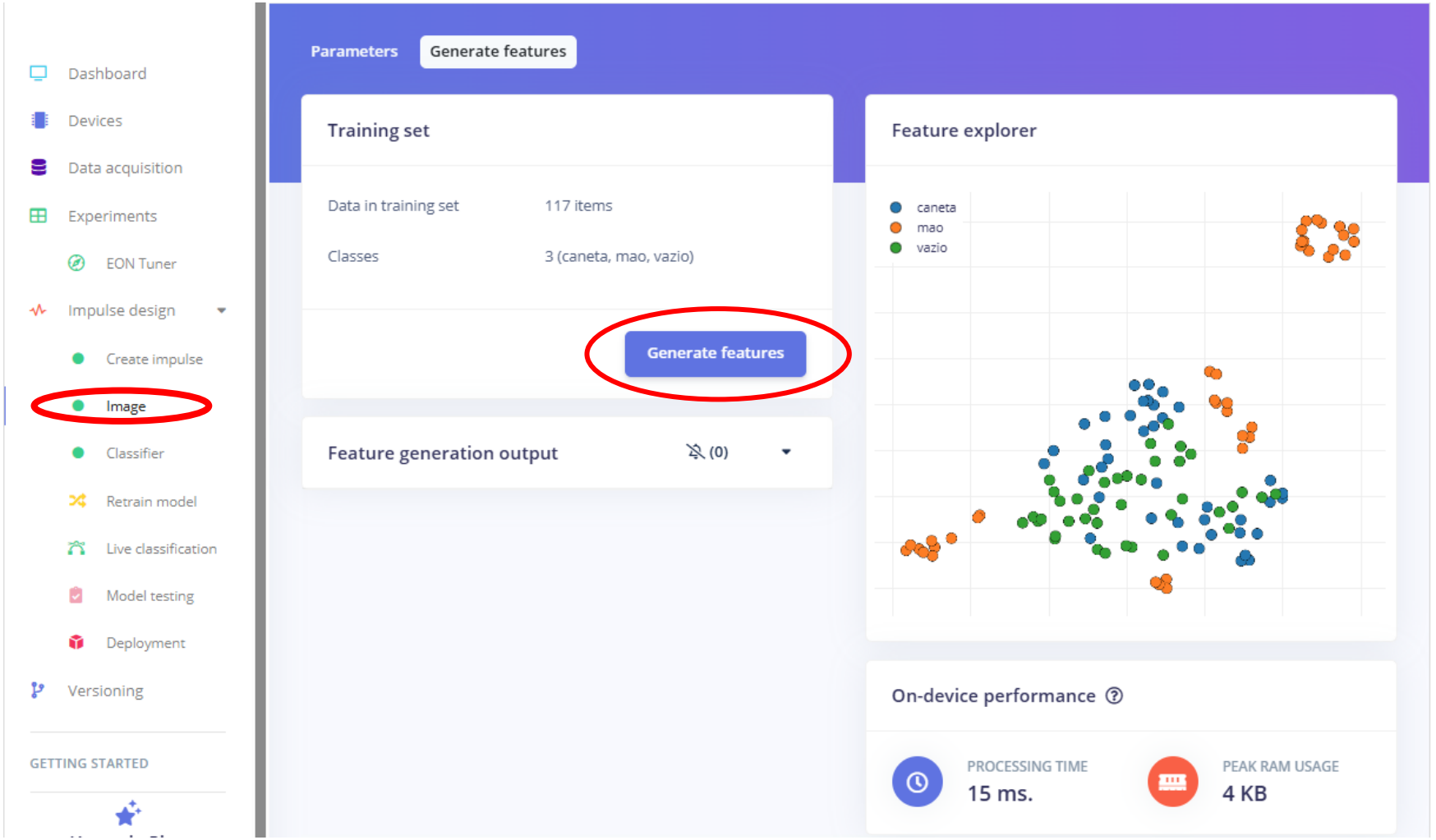
Configurar o bloco ***Image***

- *Color depth = Grayscale*
  - Tons de cinza reduzem o volume de entrada: a cor não é necessária para distinguir essas classes.



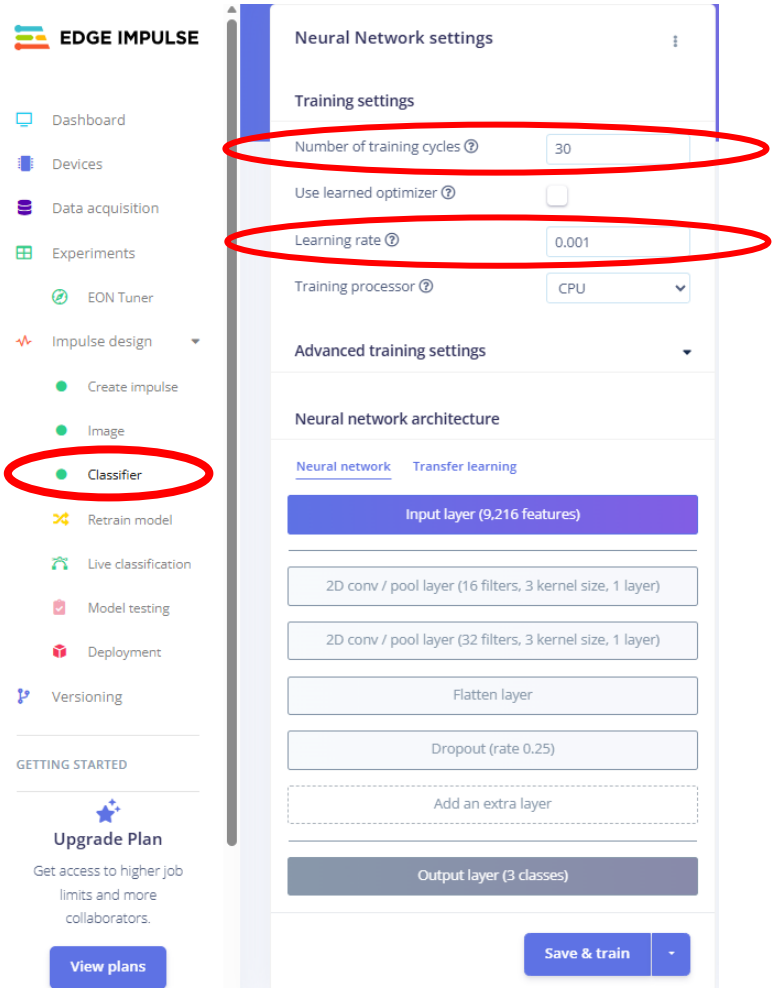
2) Edge Impulse (treino e exportação)

- Gerar as características e examinar o ***Feature Explorer***



2) Edge Impulse (treino e exportação)

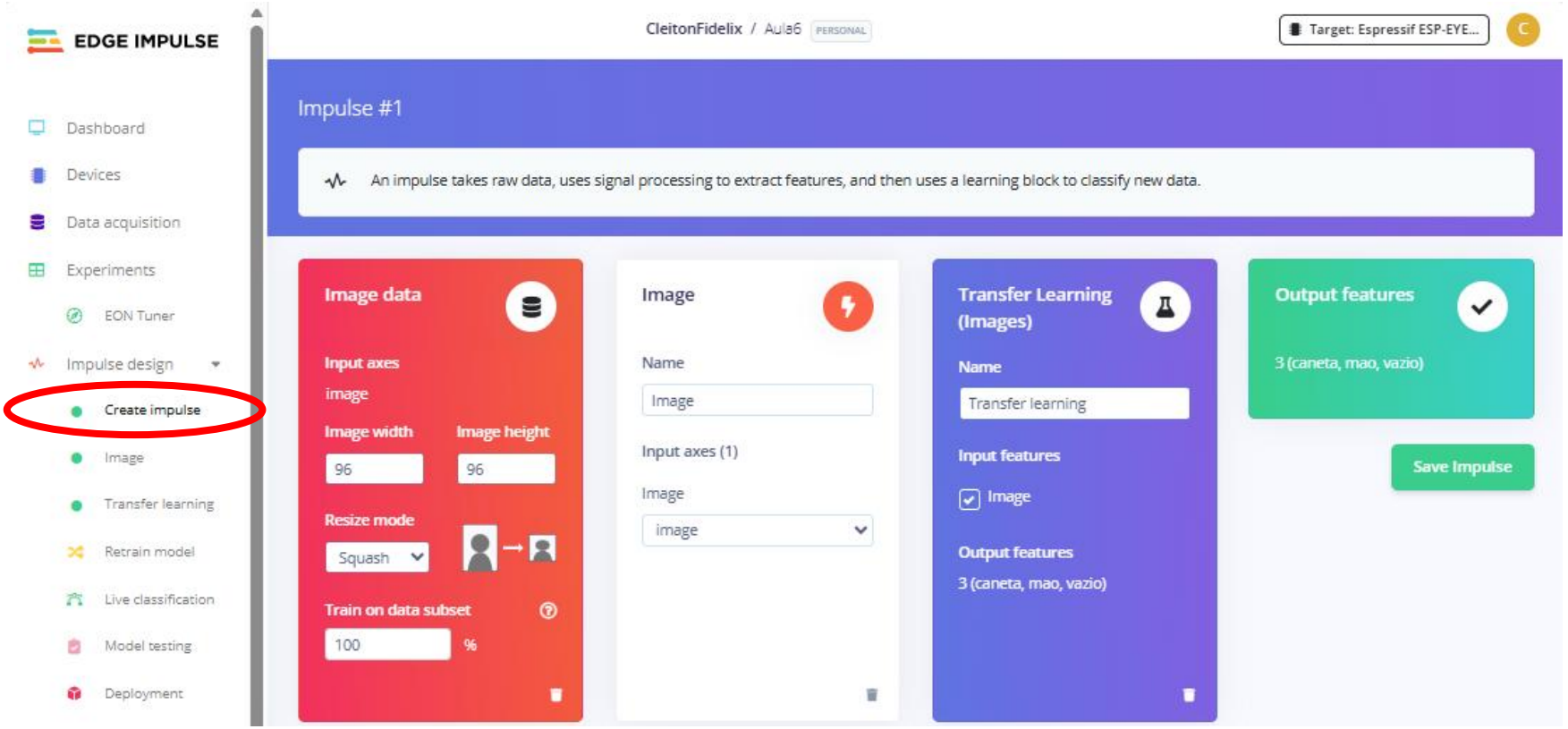
- Treinar o classificador **Classifier**



- Numero de ciclos: 30
- learning rate: 0,001
- dropout 0,25

2) Edge Impulse (treino e exportação)

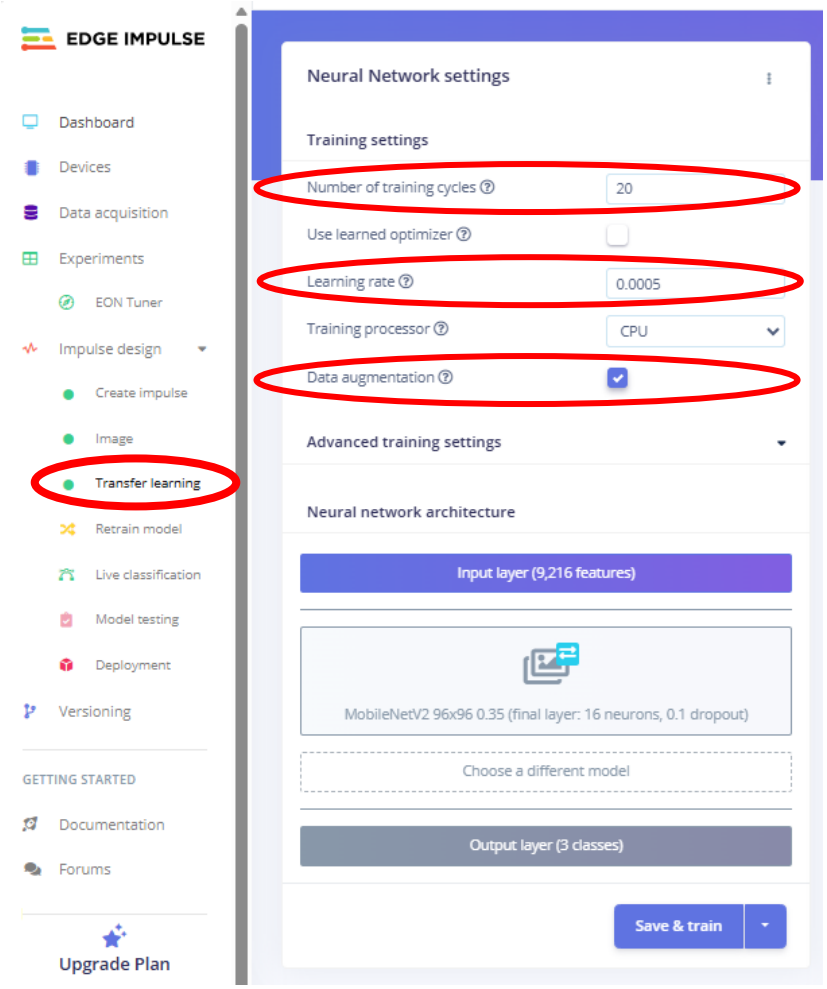
- **Opção 2:** Criar um impulso para imagem com ***Transfer learning***



2) Edge Impulse (treino e exportação)

- Treinar o classificador **Transfer learning**

- Numero de ciclos: 20
- Learning rate: 0,0005
- Data augmentation: sim
- Modelo: MobileNetV2 96x96 0.35



2) Edge Impulse (treino e exportação)

Aprendizado por transferência: ***Transfer Learning***

- Utiliza como ponto de partida uma rede neural previamente treinada com um grande conjunto de imagens.
- As primeiras camadas já aprenderam características visuais gerais, como bordas, texturas, formas e padrões.
- As camadas finais são adaptadas para reconhecer as classes específicas da aplicação.
- Pode produzir bons resultados com um *dataset* menor e reduzir o tempo necessário para o treinamento.
- O uso de uma rede pré-treinada não elimina a necessidade de imagens representativas e corretamente rotuladas.

2) Edge Impulse (treino e exportação)

Classification

- A rede é treinada a partir de pesos inicialmente aleatórios.
- A arquitetura pode ser configurada diretamente no Edge Impulse.
- Pode produzir modelos menores e oferecer maior controle sobre as camadas.
- Normalmente necessita de mais imagens para aprender características visuais robustas.

Transfer Learning

- Parte de uma rede convolucional previamente treinada.
- Reutiliza características visuais já aprendidas e adapta as camadas finais.
- É especialmente indicado para projetos de imagens com *datasets* pequenos ou médios.
- Pode exigir mais memória e maior tempo de inferência no dispositivo embarcado.

2) Edge Impulse (treino e exportação)

Qual modelo utilizar?

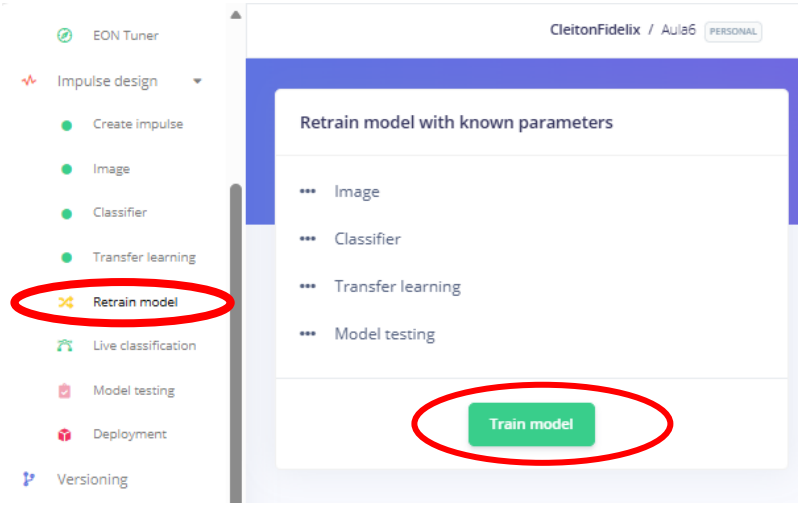
Não existe um modelo universalmente melhor. A escolha deve considerar simultaneamente:

- acurácia e F1 macro no conjunto de teste;
- Comportamento da matriz de confusão;
- Desempenho com imagens novas;
- Uso de memória RAM e flash;
- Tempo de inferência no ESP32.
- Para esta atividade, o ***Transfer Learning*** é a alternativa recomendada como modelo principal, pois o *dataset* possui somente 50 imagens por classe.

2) Edge Impulse (treino e exportação)

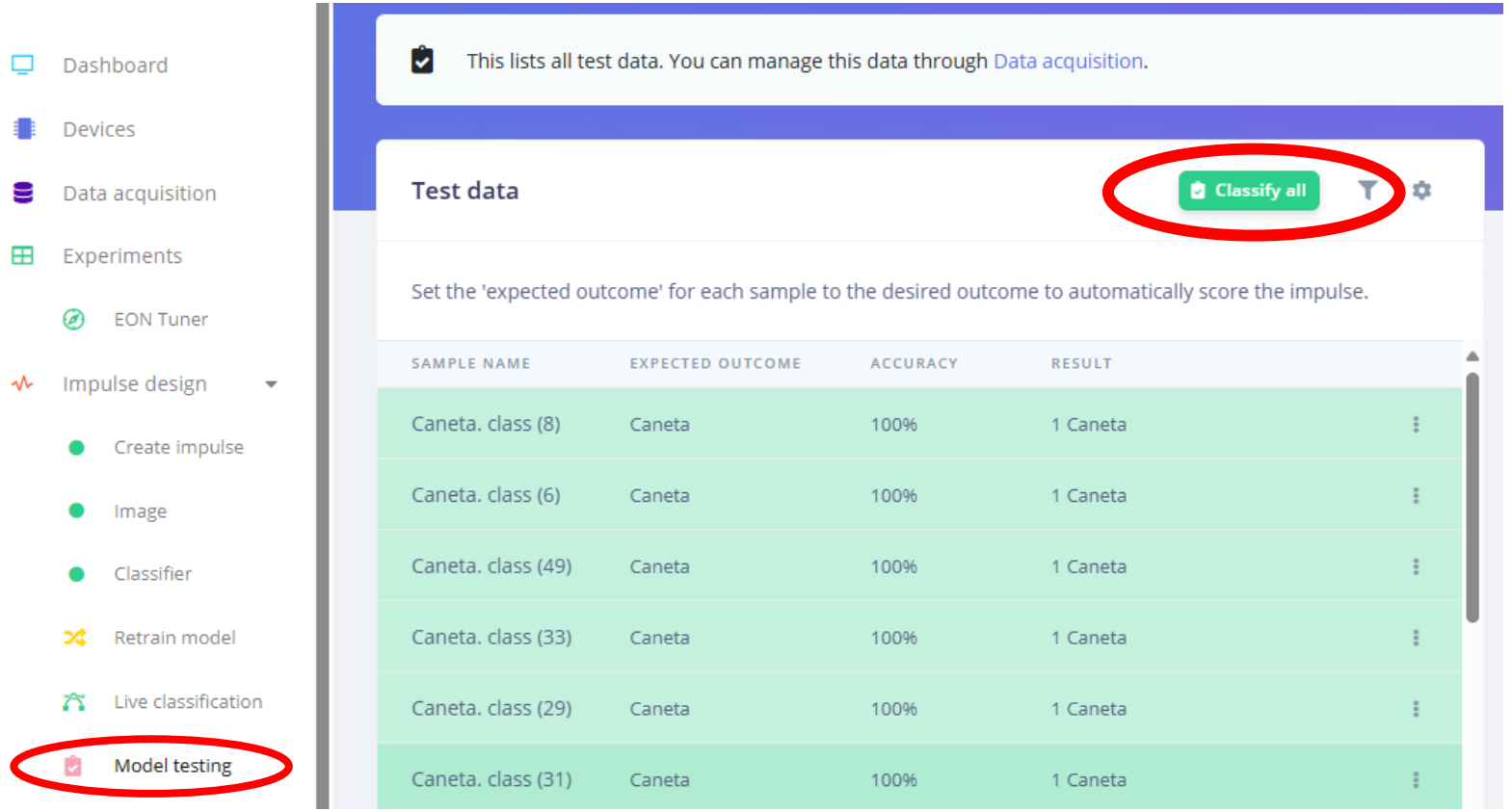
Retreinar os modelos: ***Retrain model***

- Regenera as características e treina novamente todos os blocos de aprendizado do impulso.
- Mantém as configurações já definidas para o processamento, a arquitetura e o treinamento.
- É indicado após adicionar, remover ou corrigir amostras, alterar o pré-processamento ou modificar os hiperparâmetros.
- Não é uma etapa obrigatória após o treinamento nem uma etapa adicional de ajuste fino.



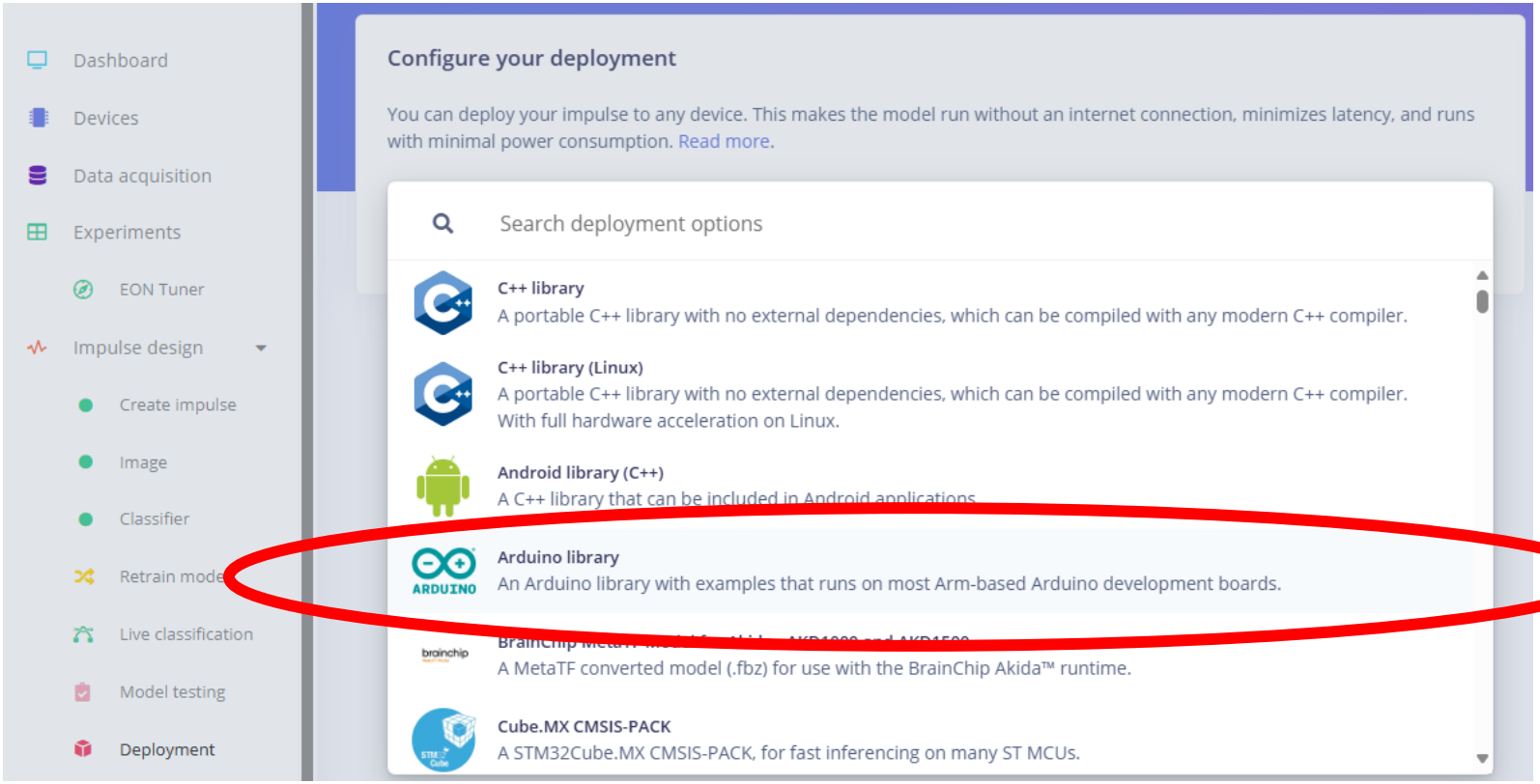
2) Edge Impulse (treino e exportação)

- Validar o modelo treinado



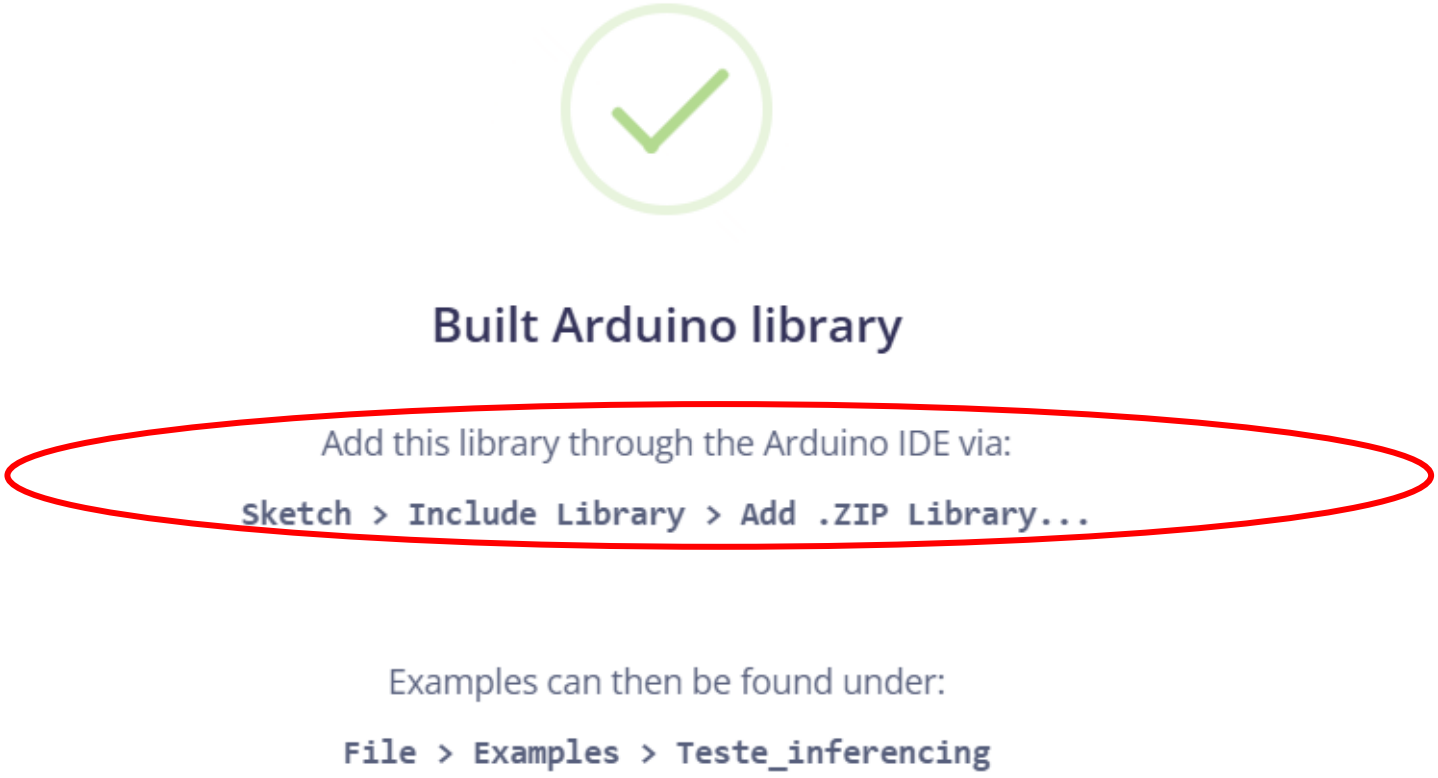
2) Edge Impulse (treino e exportação)

- Selecione **Arduino library** como formato de exportação.



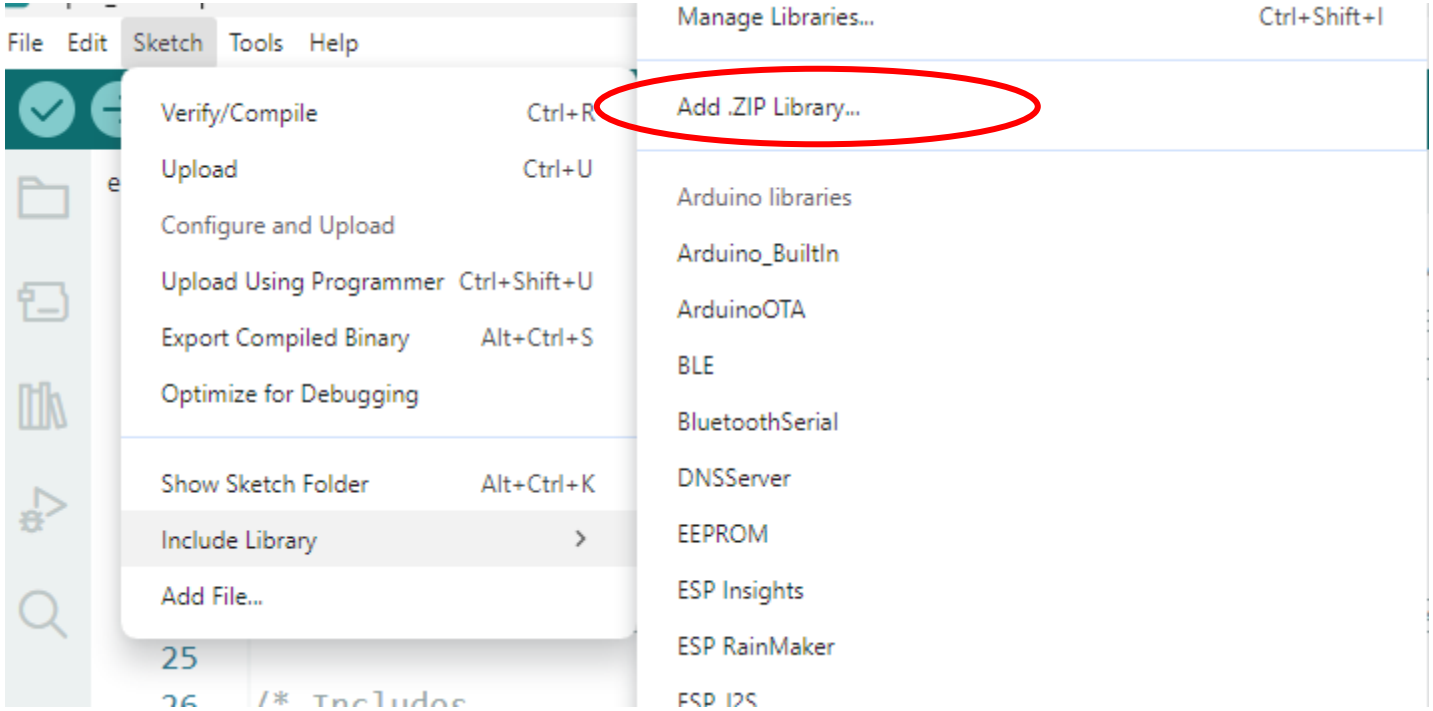
2) Edge Impulse (treino e exportação)

- Instruções para utilizar o código gerado no ambiente Arduino IDE



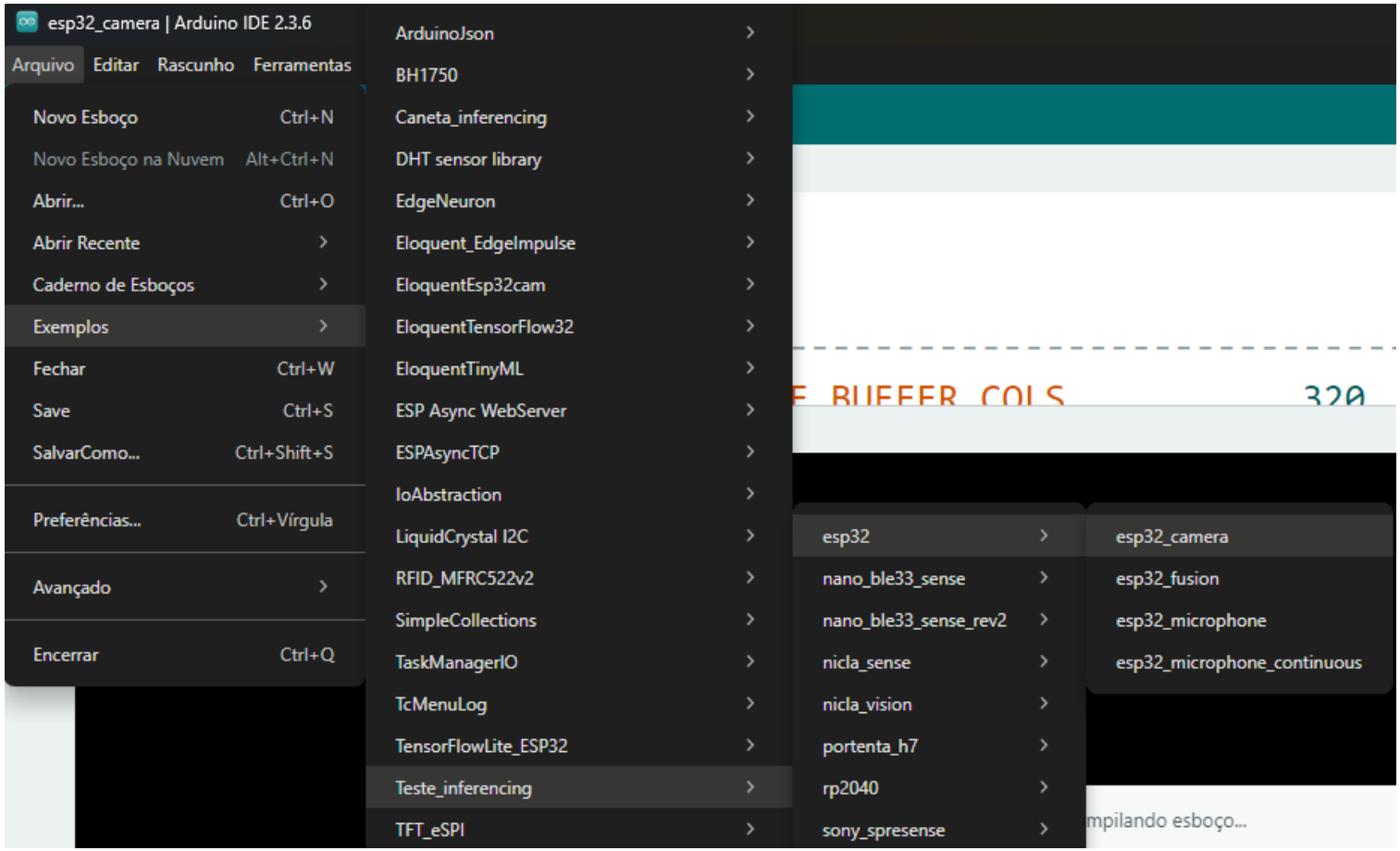
3) Arduino IDE

- Incluir a biblioteca gerada pelo Edge Impulse



3) Arduino IDE

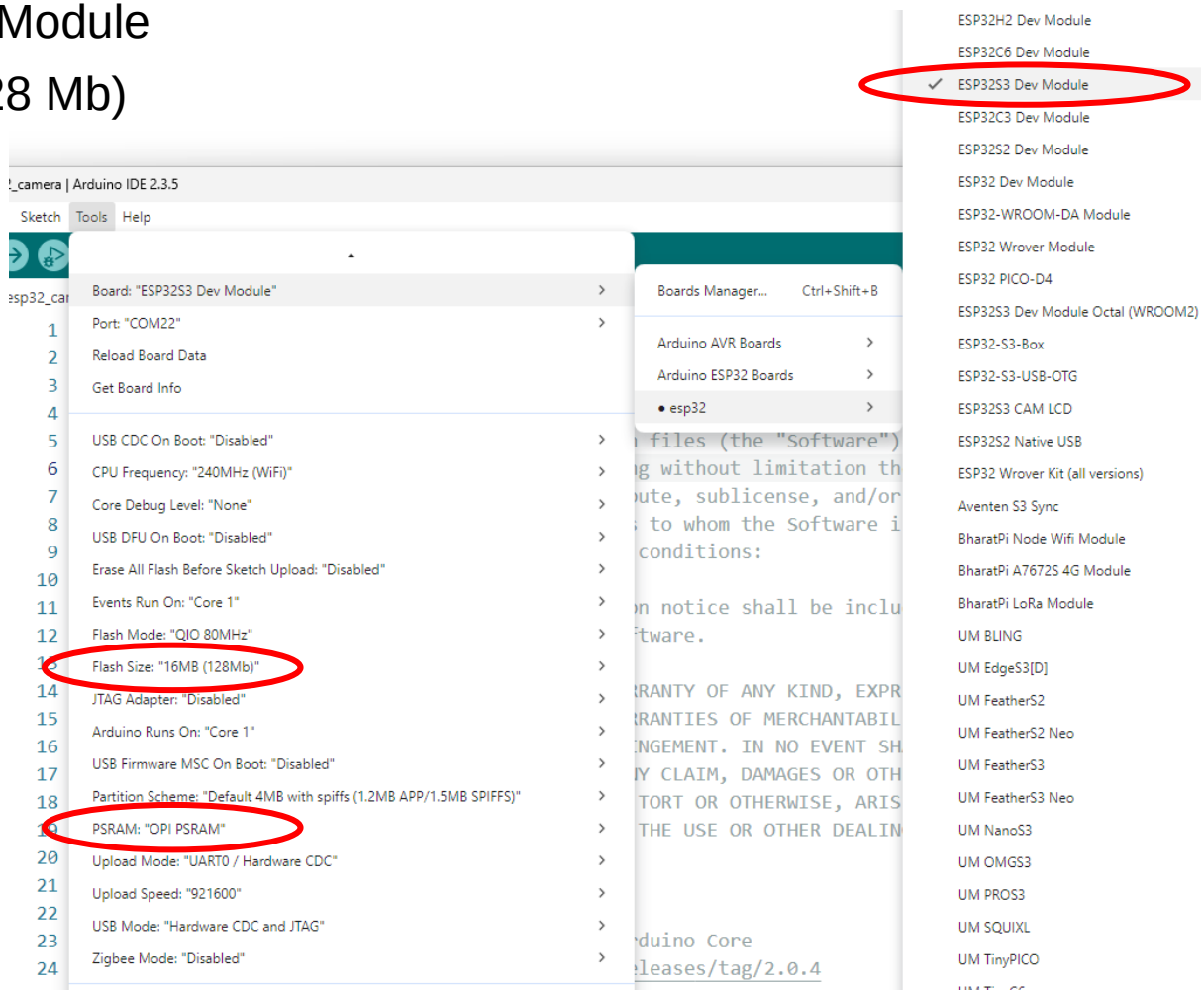
- Em exemplos, acessar o sketch **esp32_camera:**



3) Arduino IDE

Configurações para o **ESP32-S3-CAM**

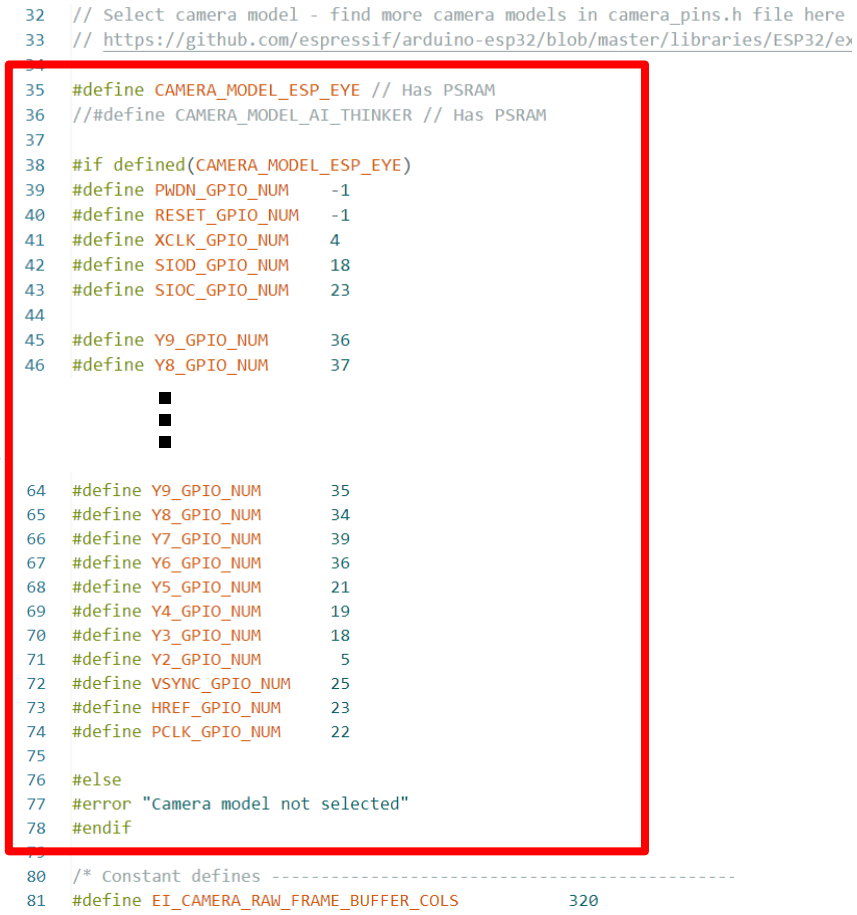
- **Placa:** ESP32S3 Dev Module
- **Flash Size:** 16 MB (128 Mb)
- **PSRAM:** OPI PSRAM
- **Porta:** *selecionar a porta correspondente



3) Arduino IDE

Substituir o código abaixo (definição dos pinos da câmera no ESP32-S3) no trecho indicado:

```
#define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
#if defined(CAMERA_MODEL_ESP32S3_EYE)
#define PWDN_GPIO_NUM  -1
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM  15
#define SIOD_GPIO_NUM  4
#define SIOC_GPIO_NUM  5
#define Y2_GPIO_NUM 11
#define Y3_GPIO_NUM 9
#define Y4_GPIO_NUM 8
#define Y5_GPIO_NUM 10
#define Y6_GPIO_NUM 12
#define Y7_GPIO_NUM 18
#define Y8_GPIO_NUM 17
#define Y9_GPIO_NUM 16
#define VSYNC_GPIO_NUM 6
#define HREF_GPIO_NUM  7
#define PCLK_GPIO_NUM  13
#else
#error "Camera model not selected"
#endif
```



```
32  // Select camera model - find more camera models in camera_pins.h file here
33  // https://github.com/espressif/arduino-esp32/blob/master/libraries/ESP32/ex
34
35  #define CAMERA_MODEL_ESP_EYE // Has PSRAM
36  //#define CAMERA_MODEL_AI_THINKER // Has PSRAM
37
38  #if defined(CAMERA_MODEL_ESP_EYE)
39  #define PWDN_GPIO_NUM    -1
40  #define RESET_GPIO_NUM   -1
41  #define XCLK_GPIO_NUM    4
42  #define SIOD_GPIO_NUM    18
43  #define SIOC_GPIO_NUM    23
44
45  #define Y9_GPIO_NUM      36
46  #define Y8_GPIO_NUM      37
            ⋮
64  #define Y9_GPIO_NUM      35
65  #define Y8_GPIO_NUM      34
66  #define Y7_GPIO_NUM      39
67  #define Y6_GPIO_NUM      36
68  #define Y5_GPIO_NUM      21
69  #define Y4_GPIO_NUM      19
70  #define Y3_GPIO_NUM      18
71  #define Y2_GPIO_NUM       5
72  #define VSYNC_GPIO_NUM   25
73  #define HREF_GPIO_NUM    23
74  #define PCLK_GPIO_NUM    22
75
76  #else
77  #error "Camera model not selected"
78  #endif
79
80  /* Constant defines -------------------------------------------------------
81  #define EI_CAMERA_RAW_FRAME_BUFFER_COLS          320
```

3) Arduino IDE

- No **bool ei_camera_capture(....)**, incluir o trecho abaixo logo após **if (!fb)** e antes do **bool converted = fmt2rgb888(...)**

```
// Bloco para preview no VSCode
// Mínimo necessário
 if (Serial.available()) {
     String comando = Serial.readStringUntil('\n');
     comando.trim();
     if (comando == "f") {
         Serial.println("START_IMAGE");
         Serial.write(fb->buf, fb->len);
         // envia APENAS os bytes do JPEG
         Serial.println("END_IMAGE");
     }
 }
```

```
316  */
317  bool ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf) {
318      bool do_resize = false;
319
320      if (!is_initialised) {
321          ei_printf("ERR: Camera is not initialized\r\n");
322          return false;
323      }
324
325      camera_fb_t *fb = esp_camera_fb_get();
326
327      if (!fb) {
328          ei_printf("Camera capture failed\n");
329          return false;
330      }
331
332      bool converted = fmt2rgb888(fb->buf, fb->len, PIXFORMAT_JPEG, snapshot_buf);
333
334      esp_camera_fb_return(fb);
335
```

4) Visualização no VS Code

- Abra o arquivo **Preview_comentado.py** no VSCode

